



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Code of Conduct:

I Oviya M / 192321117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection. (Bloom's Level 3)

Answer:

Testing as an engineering activity means that testing is planned, designed, executed and measured in a disciplined way, just like any other engineering work, instead of being done by chance. The redesigned approach for the startup is as follows.

- Analyse the requirements and identify test objectives and risks.
- Design test cases using techniques such as equivalence partitioning, boundary value analysis and decision tables, each with input data and expected result.
- Maintain a requirement traceability matrix so that every requirement has at least one test case.
- Execute tests in a controlled environment, log defects with reproducible steps and measure coverage and defect trends.

Structured design improves quality because the tests are repeatable, cover all requirements systematically and avoid duplication. For example, for an age field that accepts 18 to 60, random testing may try only a few valid ages, whereas structured design deliberately selects 17, 18, 19, 59, 60 and 61 and also invalid data such as letters, so boundary defects are not missed. Results become consistent and defect detection becomes measurable.

2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples. (Bloom's Level 3)

Answer:

A well-defined testing process consists of test planning and control, test analysis and design, test environment setup, test execution, defect reporting and tracking, and test closure, with entry and exit criteria for every stage.

In a banking system this ensures that critical functions such as fund transfer, balance enquiry and bill payment are tested in a planned manner. For example, a fund transfer test suite would include insufficient balance, daily transfer limit, network failure in the middle of a transaction (rollback) and two simultaneous transfers from the same account.

Benefits: (i) high-risk areas are tested first; (ii) failures are caught before production, which protects money and customer trust; (iii) results are repeatable and auditable, which is important for banking regulations; (iv) defects are tracked to closure; and (v) the effort and cost of testing become predictable. The overall effect is fewer production failures and higher reliability of real-time transactions.

3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes. (Bloom's Level 3)

Answer:

Testing should run in parallel with every SDLC phase, as in the V-model.

- Requirements: review requirements and prepare the acceptance test plan.
- Design: review the design and prepare system and integration test cases.
- Coding: perform unit testing, code reviews and static analysis.
- Integration: test interfaces between modules, for example cart to payment gateway to order management.
- System testing: test the complete platform for functionality, performance, security and compatibility.
- Deployment: user acceptance testing and smoke testing; Maintenance: regression testing.

Process discipline, through entry and exit criteria, documented test cases, traceability and defect tracking, ensures that no phase is skipped and that defects are found in the phase where they occur. For example, checking the payment interface during integration testing prevents order and payment mismatch from reaching the system test or the customer. This reduces rework and gives consistent, predictable quality.

4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples. (Bloom's Level 3)

Answer:

Error (mistake): a human action that produces an incorrect result, for example a developer writes the condition `length > 8` instead of `length >= 8`.

Defect (bug or fault): the flaw in the code or document caused by the error, that is, the wrong condition in the program.

Failure: the visible deviation of the system from the expected behaviour when the defect is executed, for example a user with an 8-character valid password is rejected.

Classification: the mismatch between expected and actual output observed during validation is a failure. The wrong login validation logic behind it is the defect, and the developer's mistake that introduced it is the error. The relationship is: Error leads to Defect, and Defect (when executed) leads to Failure. Not every defect causes a failure, since a defect in code that is never executed remains hidden.

5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example. (Bloom's Level 3)

Answer:

The principle applied is "Early Testing" (shift-left): testing activities should start as early as possible in the lifecycle, beginning with requirements and design reviews, and not only after coding.

It improves effectiveness because the cost of correcting a defect rises sharply with every phase in which it stays undetected. A defect found during requirement review may need only a document change, but the same defect found after release needs redesign, recoding, retesting and redeployment.

Example: if the requirement states "the account is locked after several failed attempts", an early review will ask how many attempts. Fixing this ambiguity in the requirement phase costs minutes, whereas finding that the login module locks accounts at the wrong count after release costs much more. Early testing, together with test design during the requirement phase, therefore improves defect detection efficiency and reduces overall cost.

6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement. (Bloom's Level 3)

Answer:

A tester contributes in the requirement phase by:

- reviewing the SRS for ambiguity, incompleteness, contradictions and testability;

- asking clarifying questions and raising queries on vague terms such as "quickly" or "appropriate";
- defining acceptance criteria and measurable conditions for each requirement;
- building a requirement traceability matrix and starting test design early.

Example: the requirement "the system shall calculate the medicine dosage appropriately" is unclear. A tester would ask for the formula, units, age and weight ranges and maximum limits, which makes the requirement testable.

Importance of early involvement: ambiguities are removed before they turn into design and code defects, rework cost is reduced, and in a healthcare system it also protects patient safety, because wrong dosage or record handling can be life threatening.

7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues. (Bloom's Level 3)

Answer:

Defects usually originate from a human error in an early phase and then propagate. In the payroll system the origin is the requirement phase: an ambiguous statement such as "overtime is paid extra" does not state the rate, the threshold hours or whether holidays are included. The designer then interprets it in one way, the developer codes another assumption, and the tester, who has the same unclear document, writes tests based on a third assumption. The defect is therefore carried through design, code and test, and appears as incorrect salary output in production.

Requirement clarity prevents this through: unambiguous and measurable statements (for example, "overtime beyond 8 hours per day is paid at 1.5 times the hourly rate"); worked examples and acceptance criteria; requirement reviews and walkthroughs with the stakeholders; and traceability of every requirement to design and test cases. When requirements are clear, all team members share the same understanding, and defects are prevented at the source.

8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. (Bloom's Level 3)

Answer:

Classification: this is a functional (logic or input-handling) defect of the boundary-condition type, with high severity because it causes a crash. It is justified because the system misbehaves only at the limits of the valid input range, which commonly happens when a developer writes < instead of <= or makes an off-by-one error, and because the

system does not handle the input as per the requirement.

Boundary Value Analysis (BVA) tests values at and around the edges of each input range, where defects are most likely. For a field accepting values from 1 to 100, BVA selects 0, 1, 2, 99, 100 and 101. Running these tests would have exposed the crash for the boundary values before release, and the missing coverage of the edges would have been closed. BVA gives high defect detection with only a small number of test cases.

9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality. (Bloom's Level 3)

Answer:

A defect repository stores details of each defect such as module, type, severity, phase of detection, root cause and fix. It can support test design as follows.

- Analyse patterns to find defect-prone modules and defect types, and design more test cases for them.
- Convert every fixed defect into a regression test case so that it does not reappear.
- Build checklists and error-guessing lists from common historical defects.
- Use metrics such as defect density and defect trends to plan and prioritise testing.

Example: if the repository shows repeated date-format defects in each release, the team adds test cases for different date formats, leap years and time zones to the standard suite. The repository therefore acts as organisational knowledge: it prevents repeated defects, improves test coverage, supports root cause analysis and steadily improves software quality.

10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented. (Bloom's Level 3)

Answer:

A functional defect exists when the software does not behave as specified in the functional requirements. In this ATM, the withdrawal function fails because of wrong transaction logic. For example, the account is debited but cash is not dispensed, or a withdrawal larger than the balance is allowed, or the balance is not rolled back when the transaction is interrupted. The input and output of the function do not match the specified behaviour, so this is a functional defect.

It could have been prevented by: clear requirement reviews of the withdrawal rules; test cases from decision tables and state transition testing (card inserted, PIN verified, amount entered, cash dispensed); boundary tests on limits and balance; negative tests such as power

or network failure mid-transaction; unit testing and code reviews of the transaction logic; and regression testing after each change.

11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. (Bloom's Level 3)

Answer:

Defect clustering is the testing principle that a small number of modules contain most of the defects (the Pareto 80/20 rule). To set priorities, the team analyses defect data from earlier releases and testing, and identifies modules with high defect density, high complexity, frequent changes or many dependencies. These modules are given the highest priority: more test cases, deeper techniques, more frequent regression runs, code reviews and early testing.

This improves efficiency because limited time and effort are spent where the probability of finding defects is highest, so more defects are found with fewer resources. Low-risk modules receive lighter testing. The team should still review the clusters regularly, because defects can move and repeating the same tests reduces their effectiveness (pesticide paradox).

12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. (Bloom's Level 3)

Answer:

Defects can be classified by type (functional, performance, usability, security, compatibility, interface), by severity and priority, by the phase in which they were injected, and by root cause. Slow response under heavy load is a non-functional defect of the performance (scalability) type. It is usually of high severity and high priority because it affects many users, and possible causes include unoptimised database queries, missing caching, memory leaks, thread contention or insufficient server capacity.

Classification helps resolution because the defect is assigned to the right team (for example database or infrastructure engineers), the correct test types (load, stress and endurance testing) and tools are used to reproduce it, priorities are set correctly, and trend analysis shows which defect types recur, so that preventive action can be taken.

13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency. (Bloom's Level 3)

Answer:

Historical defect data can be used in regression testing as follows.

- Every earlier defect that was fixed is converted into a regression test to confirm that it does not return.
- Modules with many past defects are identified and given more regression coverage.
- Tests are prioritised by the severity and frequency of past defects and by the impact of the present change.
- Redundant tests that never found defects are removed, and stable, repeatable tests are automated.

This improves coverage because areas that have actually failed before are always retested, and it improves efficiency because the team runs a smaller, risk-based suite instead of everything, which saves time while keeping the chance of detecting regressions high.

14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps. (Bloom's Level 3)

Answer:

The tester's role in minimising such defects includes: taking part in requirement and design reviews; understanding module interfaces and data contracts early; sharing test plans, test cases and test data with developers; writing clear, reproducible defect reports with steps, expected and actual results; participating in daily meetings and defect triage; and designing integration and API tests while the modules are still under development.

Early involvement helps because interface mismatches, such as differences in data format, parameters or sequence of calls, are identified during design instead of during integration. Developers and testers share the same understanding of requirements, misunderstandings are cleared quickly, and fewer defects reach integration testing, so the cost and delay of rework are reduced.

15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. (Bloom's Level 3)

Answer:

Defect prevention aims at removing the causes of defects, not just fixing the symptoms. The techniques are:

- Root cause analysis using Pareto charts, fishbone diagrams and the 5 Whys, to find why the defects recur.
- Corrective and preventive actions such as coding standards, design and code checklists, peer reviews and inspections.

- Training developers and testers on the areas that cause defects.
- Static analysis tools, requirement templates and automated regression tests for known defect types.
- Periodic causal analysis meetings and process improvement using lessons learned.

Prevention improves quality because fewer defects are injected in the first place, the cost of testing and rework falls, release cycles become stable and customers see more reliable software. Quality moves from inspection after the fact to building quality into the process.

16. A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. (Bloom's Level 3)

Answer:

The redesigned strategy applies the following testing principles: early testing; risk-based prioritisation, because exhaustive testing is impossible; defect clustering, to focus on crash-prone modules; updating tests regularly to avoid the pesticide paradox; and testing in context, since mobile apps depend on device conditions.

The standard strategy would define test levels (unit, integration, system, acceptance), a device and OS version matrix, test cases for interruptions such as calls and notifications, low memory and battery, changing network conditions and orientation, crash-log analysis, automated regression in a CI pipeline, standard templates and exit criteria.

Structured approaches improve reliability because every release is tested in the same, repeatable way, risky scenarios that cause crashes are covered, defects are found before release and the results can be measured and compared, which gives stable, dependable behaviour on real devices.

17. A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality. (Bloom's Level 3)

Answer:

Testing is the process of executing the software, or reviewing work products, to find failures and check that requirements are met. It is planned and done mainly by testers. Debugging is the process of finding the exact cause of a failure in the code and fixing it. It is done by developers.

Scenario: in a ride-booking app, a tester applies a discount coupon and the payment fails. The tester logs the defect with steps, expected and actual results (testing). The developer reproduces it, traces the code, finds that a null coupon value is not handled and corrects it (debugging). The tester then retests the fix and runs regression tests (testing again).

Roles: testing reveals that a problem exists and shows its impact; debugging removes its cause. Neither can replace the other. Together, with the cycle of test, debug and retest, they ensure defects are found quickly and fixed correctly, which improves software quality.

18. An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. (Bloom's Level 3)

Answer:

Strategy: (1) use analytics to select the target browsers, versions, operating systems and devices (for example Chrome, Firefox, Safari, Edge, on desktop and mobile); (2) prepare a compatibility matrix, prioritised by risk and usage, following the principle that exhaustive testing is impossible; (3) design test cases for layout, fonts, CSS, JavaScript, forms, cart, checkout and payment flows; (4) execute manual tests on key combinations and automated tests with tools such as Selenium or BrowserStack; (5) log browser-specific defects with screenshots and environment details; and (6) repeat as regression on every release.

Handling: validate HTML and CSS against standards, use progressive enhancement and responsive design, avoid browser-specific code or add proper fallbacks, and fix defects by priority. This gives a consistent shopping experience on all supported browsers and reduces customer dissatisfaction.

19. A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability. (Bloom's Level 3)

Answer:

Gaps can be identified by reviewing requirements and the existing tests against each input and business rule. Typical validation gaps in a logistics application are: missing mandatory-field checks; wrong data type or format (for example pincode, phone number, date); out-of-range values (negative weight or zero quantity); inconsistent cross-field rules (weight exceeding vehicle capacity, delivery date earlier than the dispatch date); and duplicate shipment entries.

Techniques such as equivalence partitioning, boundary value analysis, negative testing and a traceability matrix are applied to cover these gaps. Proper testing improves reliability because invalid data is rejected or handled gracefully at the entry point, the same input always gives the same, correct output, and wrong data does not move downstream into routing, billing or tracking.

20. A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design. (Bloom's Level 3)

Answer:

Applying testing as an engineering activity means that the team first analyses requirements, identifies test conditions, designs test cases with techniques (equivalence partitioning, boundary value analysis, decision tables, state transition), prepares test data and expected results, links the tests to requirements, and only then executes and measures them.

Importance of systematic test design: it ensures all requirements and risks are covered, avoids duplicate tests, finds more defects with fewer test cases, makes tests repeatable and reusable for regression, helps new team members, and provides measurable coverage. For example, a login screen tested randomly may miss an empty password or a locked account, whereas a designed set of test cases covers valid, invalid, empty, boundary and locked-account conditions. Random testing leaves these to chance.

21. A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency. (Bloom's Level 3)

Answer:

Root causes are identified by tracing the data flow between modules and analysing defects with integration testing. Typical causes are: mismatched data formats or units between modules (mg versus g, date formats); interface or API contract mismatch; missing or delayed synchronisation, so that an update in registration is not seen in billing or pharmacy; different validation rules in different modules; duplicate or stale records; and transaction failures without rollback. Principles applied are early testing of interfaces, risk-based testing of critical patient data and defect clustering around the integration points.

Proper testing ensures consistency through interface and integration tests, end-to-end scenarios (admission, treatment, billing), database verification and data reconciliation between modules, negative tests for failures in the middle of an update, and regression tests after every change. This keeps patient data accurate and identical in all modules.

22. A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality. (Bloom's Level 3)

Answer:

Role of testers across the SDLC:

- Requirements: review for clarity and testability, define acceptance criteria.
- Design: review the design, identify risks and prepare the test plan and test cases.
- Coding: support unit testing, static analysis and code reviews, and prepare test data and environment.

- Testing: execute integration, system and regression tests and report defects.
- Deployment: user acceptance and smoke testing. Maintenance: regression and retesting of fixes.

Early involvement improves quality because ambiguities and design flaws are found before coding, test cases are ready when the build arrives, so test execution starts without delay, and developers and testers work with the same understanding. Defects are cheaper to fix, there is less rework, and the schedule is better controlled.

23. A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures. (Bloom's Level 3)

Answer:

The gaps are found by comparing the existing tests with the requirements, usage profile and risks. The principles applied are: exhaustive testing is impossible, so testing is based on risk; testing depends on context, so the real peak usage patterns must be included; and defects cluster, so heavily used modules are tested more. Typically missing are non-functional scenarios: load, stress, spike and endurance tests, concurrent users, large data volumes and failure recovery.

Better coverage, meaning requirement coverage, risk coverage and scenario coverage, prevents failures because the system is tried under realistic and extreme conditions before release. For example, a flash-sale simulation with thousands of simultaneous users exposes database bottlenecks and timeouts in test, where they can be fixed, and not in production during peak usage.

24. A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis. (Bloom's Level 3)

Answer:

The team should set up a central defect repository using a tool such as Jira or Bugzilla. Each defect record should contain a unique ID, title, description, steps to reproduce, expected and actual result, environment, module, severity, priority, phase of detection, status, assigned person, root cause and attachments such as screenshots or logs. A defect life cycle should be followed: New, Assigned, Open, Fixed, Retest, Verified and Closed, with Reopened, Rejected or Deferred where applicable.

Documentation enhances tracking because every defect has an owner and a status, so none is lost or forgotten, and developers can reproduce the problem easily. It enhances analysis because the data can be used for defect density, trends, defect age, clusters and root causes, which supports better planning, test design and prevention of repeated defects.

25. A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability. (Bloom's Level 3)

Answer:

The redesigned approach applies the principles of early testing, risk-based testing (high-value transactions first), defect clustering, regular updating of tests and testing in context. It includes a documented test plan and strategy; test cases for the properties of a transaction (atomicity, consistency, isolation and durability); boundary tests on amounts and limits; negative tests such as insufficient balance and network failure; concurrency tests for simultaneous transactions; security tests; end-to-end reconciliation of ledgers; and automated regression with entry and exit criteria.

Structured testing enhances reliability because each critical transaction path is verified systematically and repeatedly, failures such as double debit or lost transactions are found before release, defects are tracked until closure, and the quality of every release can be measured. The result is stable and trustworthy banking operations.

26. A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection. (Bloom's Level 3)

Answer:

Normal inputs only confirm that the system works under typical conditions. To include edge cases, the principles of equivalence partitioning, boundary value analysis, error guessing and risk-based testing are applied. Edge cases include minimum and maximum values, just inside and just outside the limits, zero, negative, empty or null values, very long strings, special characters, leap-year dates and maximum file size.

Boundary testing improves defect detection because programmers often make off-by-one mistakes or wrong relational operators at limits, so defects are concentrated there. For a field that accepts 1 to 50, testing 0, 1, 2, 49, 50 and 51 detects such mistakes with only a few test cases, while a normal value such as 25 would never reveal them.

27. A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes. (Bloom's Level 3)

Answer:

A process-based approach defines one common test process for all modules. It includes a test policy and test strategy; standard templates for the test plan, test cases and defect reports; defined entry and exit criteria; the same test design techniques and coverage targets; peer reviews of test cases; common metrics such as defect density and test

coverage; and periodic process improvement (for example using TMMi or lessons learned). Standardisation improves outcomes because every module is tested to the same depth, regardless of who tests it, the results are comparable and repeatable, weak areas are easy to see from the metrics, new members adapt quickly and quality becomes even across the application.

28. A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness. (Bloom's Level 3)

Answer:

Input validation is improved by adding negative and invalid-input tests using equivalence partitioning, boundary value analysis, error guessing and fuzz testing. Scenarios include empty and null fields, wrong data types, very long text, special characters, script or SQL injection strings, invalid dates, duplicate submissions and file upload of wrong type or size. Validation should be tested on both the client and the server side.

Proper testing ensures robustness because the application is shown to reject or handle bad input gracefully, with clear error messages, without crashing, exposing data or corrupting the database. For example, entering a script tag in a comment box should be sanitised and not executed. A robust system keeps working correctly even under unexpected or malicious input.

29. A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage. (Bloom's Level 3)

Answer:

Defect leakage is the percentage of defects that escape testing and are found in production:
$$\text{Defect Leakage} = (\text{defects found in production} / (\text{defects found in testing} + \text{defects found in production})) \times 100.$$

Root cause analysis, based on the principles of early testing, defect clustering, risk-based testing and context dependence, usually shows causes such as incomplete requirement coverage, missing negative or boundary scenarios, weak regression testing, a test environment or data different from production, time pressure, no user acceptance testing and poor defect analysis.

Improved practices that prevent leakage are: reviews and early testing; traceability to ensure all requirements are covered; stronger test design with boundary and negative cases; automated regression; a production-like environment and realistic data; strict exit criteria before release; and post-release defect analysis to update the test suite. Together these find more defects before release and reduce leakage.

30. A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. (Bloom's Level 3)

Answer:

The strategy is redesigned using the principles of early testing, defect clustering, risk-based testing and regular updating of tests. It includes: static testing (reviews, walkthroughs and inspections of requirements and design); test design started in the requirement phase; unit testing and code reviews by developers; continuous integration with automated tests on every build; focus on high-risk and defect-prone modules; and clear exit criteria for every phase.

Early defect detection improves efficiency because the cost of fixing a defect increases greatly in later phases. Defects found early need only small changes, avoid rework across design, code and tests, shorten the test cycle, prevent last-minute delays and reduce the number of defects reaching production. Overall cost and effort fall while quality rises.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary (5 Marks)	Level 3 - Proficient (4 Marks)	Level 2 - Developing (2-3 Marks)	Level 1 - Beginning (0-1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand